

Problem Statement

Shortest Path in a Grid Maze using Breadth-First Search

1. Background

A maze is represented as a square grid of size $n \times n$, where each cell is either open (passable) or blocked (a wall). Starting from the top-left cell of the grid, the goal is to determine whether the bottom-right cell can be reached, and if so, to find the shortest sequence of moves (up, down, left, right) that reaches it. Since every move between adjacent cells has the same cost, the shortest path is simply the path with the fewest number of moves — which makes this a natural fit for Breadth-First Search (BFS), an algorithm that explores a graph outward in layers of increasing distance from the source.

2. Problem Definition

Given an $n \times n$ grid **maze**, where **maze[i][j] = 0** denotes an open cell and **maze[i][j] = 1** denotes a wall, determine the shortest path from cell (0, 0) to cell (n-1, n-1), moving only up, down, left, or right between orthogonally adjacent open cells. If no such path exists, report that the destination is unreachable.

3. Input

- An integer **n**, the size of the grid (n rows and n columns).
- An $n \times n$ matrix of 0s and 1s representing open cells and walls.

4. Output

- If the bottom-right cell is reachable from the top-left cell, output the shortest sequence of moves (as directions: right, left, up, down) that reaches it.
- If it is not reachable, report that no path exists.

5. Constraints

- The starting cell (0, 0) and the destination cell (n-1, n-1) are always within grid bounds.
- Movement is restricted to the four cardinal directions; diagonal movement is not allowed.
- A cell may only be entered if it is open (value 0) and lies within the grid boundaries.

- All moves have equal cost, so the shortest path is measured by the number of moves, not by any weighted distance.

6. Approach

The problem is solved using Breadth-First Search, implemented without the C++ Standard Template Library. A manual queue is maintained using two parallel arrays that store the row and column indices of discovered cells, along with front and rear index pointers to track the head and tail of the queue. Starting from cell (0, 0), each cell is dequeued and its four orthogonal neighbors are examined; any neighbor that is open and not yet visited is marked visited, has its direction of arrival recorded in a parent grid, and is enqueued. Marking a cell visited at the time it is enqueued, rather than when it is dequeued, prevents the same cell from being added to the queue more than once. Because BFS explores the grid one layer of distance at a time, the destination cell is guaranteed to be reached, if at all, along a shortest path. Once the destination is dequeued, the parent grid is walked backward from the destination to the source to reconstruct the sequence of moves, which is then reversed and reported as the answer.

7. Example

Given the following 4×4 maze (0 = open, 1 = wall):

```
0 0 1 0
1 0 1 0
0 0 0 0
0 1 1 0
```

The shortest path from (0, 0) to (3, 3) is: **right, down, down, right, right, down.**